Índice por temas

Guía rápida de nueve librerías que complementan pandas. Tablas por tareas, ejemplos breves y operaciones afines agrupadas.

Cómo usar la guía 2
Qué librería elegir 3
Datos comunes de los ejemplos 4
Versiones y documentación 5
NumPy · preparación 6
Convertir y crear arrays 7
Seleccionar, calcular y simular 9
Matplotlib · preparación 12
Crear gráficos 13
Etiquetar y exportar 15
Seaborn · preparación 17
Explorar un DataFrame 18
Crear paneles y ajustar el estilo 20
SciPy · preparación 21
Describir y contrastar muestras 22
Distribuciones y cálculo científico 24
scikit-learn · preparación 25
Preparar variables 26
Ajustar y utilizar modelos 28
Evaluar y seleccionar modelos 31
PyArrow · preparación 33
Intercambiar tablas y tipos 34
Leer Parquet y procesar conjuntos 36
DuckDB · preparación 38
Consultar pandas mediante SQL 39
Leer archivos y guardar resultados 41
SQLAlchemy · preparación 43
Conectar y consultar bases de datos 44
openpyxl · preparación 46
Abrir libros y editar celdas 47
Dar formato y preparar informes 49
Índice alfabético 51

Cómo usar la guía

Lectura: busca una tarea o un nombre en el índice. Cada fila muestra una llamada representativa, su finalidad, cuándo conviene y qué resultado produce. Los nombres enlazan a la referencia oficial.

Alcance: 238 recetas seleccionadas de nueve librerías. Es una guía práctica del ecosistema de pandas, no un inventario completo de sus APIs. No incluye funciones privadas ni llamadas obsoletas recomendadas como uso principal.

Sin duplicaciones innecesarias: operaciones afines pueden compartir fila (por ejemplo, barras verticales y horizontales). La columna «Cómo se usa» muestra una de las variantes; el nombre y la nota explican la diferencia.

Ejemplos independientes: ejecuta los datos comunes y la preparación de la librería. Cada ficha parte de ese estado; no necesitas ejecutar todas las filas anteriores. Las tablas no muestran salidas completas para mantener la consulta rápida.

Objetos y nombres: df es un DataFrame; a es un array NumPy; X son variables de entrada; y es un objetivo; fig es una figura y ax uno de sus ejes. El resto se define en la preparación de cada librería. Son nombres de variables que puedes cambiar.

Archivos: los nombres representan archivos locales de ejemplo. La preparación crea las entradas necesarias. Las fichas de escritura generan o sobrescriben sus salidas: trabaja en una carpeta de práctica.

Validación: las 238 llamadas se ejecutaron con Python 3.12 y pandas 3.0.6. Las versiones exactas aparecen en «Versiones y documentación». Hay un aviso interno de compatibilidad en sns.boxplot que se indica en su ficha.

Qué librería elegir

Necesitas…	Librería	Cómo se combina con pandas
Cálculos vectorizados y arrays numéricos.	NumPy	Extrae columnas como arrays y devuelve resultados con el índice original.
Gráficos con control sobre ejes, etiquetas y exportación.	Matplotlib	Dibuja columnas y series con control de cada elemento.
Exploración estadística directamente desde un DataFrame.	Seaborn	Recibe data=df y nombres de columnas para representar grupos.
Estadística inferencial, interpolación y optimización.	SciPy	Recibe columnas numéricas para estadística y métodos científicos.
Preparación de variables, modelos y evaluación.	scikit-learn	Usa DataFrames como X, pipelines para preparar y métricas para evaluar.
Intercambio columnar, Parquet y lectura por lotes.	PyArrow	Convierte DataFrames en tablas y procesa archivos Parquet.
SQL analítico sobre pandas y archivos locales.	DuckDB	Registra DataFrames o consulta archivos y convierte solo el resultado.
Conexiones y transacciones con bases SQL.	SQLAlchemy	Permite read_sql y to_sql con conexiones y transacciones.
Edición de libros, celdas y formatos de Excel.	openpyxl	Completa el Excel de pandas con estilos, hojas, fórmulas y gráficos.

Ruta frecuente: leer y limpiar con pandas → calcular con NumPy/SciPy → visualizar con Seaborn/Matplotlib → modelar con scikit-learn → guardar con Arrow, SQL o Excel. No necesitas todas las librerías en cada proyecto.

Datos comunes de los ejemplos

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'cliente': ['Ana','Luis','Eva']*4,
    'ciudad': ['Madrid','Sevilla','Valencia']*4,
    'unidades': [1,2,3,2,4,1,5,3,2,6,4,3],
    'precio': [10.,15.,20.,12.,18.,25.,
               11.,16.,22.,13.,19.,24.],
    'fecha': pd.date_range('2026-01-01',periods=12)
})
df['total'] = df['unidades'] * df['precio']
clientes = pd.DataFrame({
    'cliente': ['Ana','Luis','Eva'],
    'segmento': ['A','B','A']})
```

La tabla contiene 12 ventas, tres clientes, tres ciudades, cantidades, precios y fechas. total es unidades × precio. Los datos son didácticos; sirven para aprender sintaxis, no para validar modelos o sacar conclusiones estadísticas.

Instalación (terminal):

```
python -m pip install numpy pandas matplotlib seaborn scipy
python -m pip install scikit-learn pyarrow duckdb sqlalchemy openpyxl
```

Versiones y documentación

Librería	Versión probada	Portal consultado	Documentación oficial
NumPy	2.5.4	2.5	https://numpy.org/doc/stable/
Matplotlib	3.11.2	3.11.2	https://matplotlib.org/stable/
Seaborn	0.13.2	0.13.2	https://seaborn.pydata.org/
SciPy	1.18.1	1.18.0	https://docs.scipy.org/doc/scipy/
scikit-learn	1.9.1	1.9.1	https://scikit-learn.org/stable/
PyArrow	26.0.0	25.0.1	https://arrow.apache.org/docs/python/
DuckDB	1.5.6	portal stable	https://duckdb.org/docs/stable/clients/python/reference/
SQLAlchemy	2.1.4	2.1.4	https://docs.sqlalchemy.org/en/21/
openpyxl	3.1.5	3.1.3	https://openpyxl.readthedocs.io/en/stable/

Consulta: 11 de octubre de 2026. Los portales oficiales pueden mostrar un parche o una edición anterior al paquete instalado; se indica la diferencia y las llamadas se contrastan con la versión probada. Las páginas enlazadas pueden actualizarse después de esta consulta.

Compatibilidad: Seaborn 0.13.2 genera un aviso interno en boxplot con Matplotlib 3.11.2; la receta funciona. No se ha ocultado el aviso en la validación. Para otros entornos, consulta los requisitos de cada paquete.

NumPy · preparación

Cálculos vectorizados y arrays numéricos. Versión probada: 2.5.4.

Importaciones

```
import numpy as np
```

Objetos de las fichas

```
a = df['total'].to_numpy(dtype=float)
Xn = df[['unidades','precio']].to_numpy()
rng = np.random.default_rng(42)
```

np es la librería; a es un ndarray, sin índice ni nombres de columnas.

Las operaciones se alinean por posición. En pandas, en cambio, muchas se alinean por etiquetas.

Para devolver un vector a pandas: pd.Series(a, index=df.index). Solo si conserva la misma longitud y orden.

NumPy · Convertir y crear arrays

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
np.asarray	`np.asarray(df['total'],dtype=float)`	Convierte valores en un array. **Nota:** Pierde las etiquetas; no garantiza una copia.	Para pasar pandas a cálculo numérico.	**NumPy** Devuelve array NumPy.
np.array	`np.array([10,20,30],dtype=float)`	Crea un array.	Para definir valores pequeños.	**NumPy** Devuelve array NumPy.
np.arange	`np.arange(0,10,2)`	Genera valores con paso constante. **Nota:** El límite final se excluye.	Para posiciones o secuencias.	**NumPy** Devuelve array NumPy.
np.linspace	`np.linspace(0,1,5)`	Genera puntos equidistantes.	Para evaluar una curva.	**NumPy** Devuelve array NumPy.
np.zeros	`np.zeros((3,2))`	Crea una matriz de ceros.	Para reservar resultados.	**NumPy** Devuelve array NumPy.
np.full	`np.full(3,np.nan)`	Rellena un array con un valor.	Para inicializar ausencias.	**NumPy** Devuelve array NumPy.
a.shape	`a.shape`	Devuelve las dimensiones.	Para comprobar la forma.	**NumPy** Devuelve tupla.
a.dtype	`a.dtype`	Indica el tipo de los elementos.	Para revisar precisión y memoria.	**NumPy** Devuelve Float64DType.
a.astype	`a.astype('float32')`	Convierte el tipo del array.	Para reducir memoria numérica.	**NumPy** Devuelve array NumPy.
a.reshape	`a.reshape(-1,1)`	Reorganiza dimensiones.	Para convertir un vector en columna.	**NumPy** Devuelve array NumPy.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
a.ravel	`a.ravel()`	Obtiene un array plano. **Nota:** Puede compartir memoria con el original.	Para algoritmos que esperan un vector.	**NumPy** Devuelve array NumPy.
a.copy	`a.copy()`	Crea una copia independiente.	Para modificar sin afectar al origen.	**NumPy** Devuelve array NumPy.
np.concatenate	`np.concatenate([a,a[:2]])`	Une arrays por un eje. **Nota:** No alinea índices como pandas.	Para juntar lotes numéricos.	**NumPy** Devuelve array NumPy.
np.column_stack	`np.column_stack([a,a*2])`	Apila vectores como columnas.	Para crear una matriz de variables.	**NumPy** Devuelve array NumPy.

NumPy · Seleccionar, calcular y simular

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
np.where	`np.where(a>=40,'alta','baja')`	Elige valores según una condición.	Para crear categorías binarias.	**NumPy** Devuelve array NumPy.
np.select	`np.select([a<30,a<60],['baja','media'], default='alta')`	Aplica varias reglas ordenadas. **Nota:** Gana la primera condición verdadera.	Para clasificar importes.	**NumPy** Devuelve array NumPy.
np.clip	`np.clip(a,20,80)`	Limita valores a un intervalo.	Para aplicar topes numéricos.	**NumPy** Devuelve array NumPy.
np.isnan	`np.isnan(a)`	Detecta NaN. **Nota:** En datos mixtos usa pd.isna().	Para revisar ausencias numéricas.	**NumPy** Devuelve array NumPy.
np.isfinite	`np.isfinite(a)`	Detecta valores finitos.	Para excluir infinitos y NaN.	**NumPy** Devuelve array NumPy.
np.nan_to_num	`np.nan_to_num([1,np.nan],nan=0)`	Sustituye valores no finitos. **Nota:** También trata infinitos por defecto.	Para preparar un cálculo.	**NumPy** Devuelve array NumPy.
np.mean / np.nanmean	`np.nanmean([10,np.nan,30])`	Calculan la media con o sin omitir NaN. **Nota:** np.mean propaga NaN; np.nanmean los omite.	Para resumir valores numéricos.	**NumPy** Devuelve número.
np.median	`np.median(a)`	Calcula la mediana.	Para reducir el efecto de extremos.	**NumPy** Devuelve número.
np.std / np.var	`np.std(a,ddof=1)`	Calculan desviación típica o varianza. **Nota:** NumPy usa ddof=0 por defecto; pandas suele usar 1.	Para medir dispersión.	**NumPy** Devuelve número.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
np.quantile	`np.quantile(a,[0.25,0.5,0.75])`	Calcula cuantiles.	Para obtener umbrales robustos.	**NumPy** Devuelve array NumPy.
np.average	`np.average(a,weights=np.ones(len(a)))`	Calcula una media ponderada.	Para dar pesos a observaciones.	**NumPy** Devuelve número.
np.sum / np.cumsum	`np.cumsum(a)`	Suman valores o calculan acumulados.	Para totales y saldos progresivos.	**NumPy** Devuelve array NumPy.
np.diff	`np.diff(a)`	Resta valores consecutivos.	Para estudiar cambios.	**NumPy** Devuelve array NumPy.
np.round	`np.round(a/3,decimals=2)`	Redondea a decimales.	Para presentar resultados.	**NumPy** Devuelve array NumPy.
np.log1p	`np.log1p(a)`	Calcula logaritmo de uno más el valor.	Para transformar importes no negativos.	**NumPy** Devuelve array NumPy.
np.isclose	`np.isclose(a,30,rtol=1e-5)`	Compara con tolerancia.	Para comparar números decimales.	**NumPy** Devuelve array NumPy.
np.unique	`np.unique(a,return_counts=True)`	Obtiene valores únicos y frecuencias.	Para revisar repeticiones.	**NumPy** Devuelve tupla.
np.argsort	`np.argsort(a)`	Devuelve posiciones de ordenación.	Para ordenar arrays relacionados.	**NumPy** Devuelve array NumPy.
np.isin	`np.isin(a,[20,30])`	Comprueba pertenencia a un conjunto.	Para construir máscaras.	**NumPy** Devuelve array NumPy.
np.corrcoef	`np.corrcoef(a,df['unidades'])`	Calcula una matriz de correlación.	Para comparar vectores.	**NumPy** Devuelve array NumPy.
np.random.default_rng	`np.random.default_rng(42)`	Crea un generador aleatorio.	Para repetir simulaciones.	**NumPy** Devuelve Generator.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
rng.choice	`rng.choice(a,size=3,replace=False)`	Extrae una muestra aleatoria.	Para seleccionar sin repetición.	**NumPy** Devuelve array NumPy.
rng.normal	`rng.normal(loc=0,scale=1,size=5)`	Genera valores normales.	Para simular ruido.	**NumPy** Devuelve array NumPy.
np.linalg.lstsq	`np.linalg.lstsq(Xn,a,rcond=None)`	Resuelve mínimos cuadrados. **Nota:** No añade intercepto automáticamente.	Para ajustar una relación lineal.	**NumPy** Devuelve tupla.

Matplotlib · preparación

Gráficos con control sobre ejes, etiquetas y exportación. Versión probada: 3.11.2.

Importaciones

```
import matplotlib.pyplot as plt
```

Objetos de las fichas

```
fig, ax = plt.subplots(
    figsize=(7,4), layout='constrained')
resumen = df.groupby('cliente')['total'].sum()
corr = df[['unidades','precio','total']].corr()
```

fig es la figura completa; ax es un eje o panel.

Usa un eje nuevo para cada ejemplo. Las funciones de dibujo añaden elementos al eje existente.

Las recetas producen objetos gráficos; los archivos solo se crean al exportarlos.

Matplotlib · Crear gráficos

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
plt.subplots	`fig,ax=plt.subplots(figsize=(7,4),layout='constrained')`	Crea una figura y sus ejes.	Para empezar un gráfico.	**Matplotlib** Asigna una Figure a fig y un Axes a ax.
ax.plot	`ax.plot(df['fecha'],df['total'],marker='o',label='Ventas')`	Dibuja una línea.	Para seguir ventas en el tiempo.	**Matplotlib** Modifica el gráfico; devuelve lista.
ax.bar / ax.barh	`ax.bar(resumen.index,resumen.values)`	Dibujan barras verticales u horizontales.	Para comparar totales por cliente.	**Matplotlib** Modifica el gráfico; devuelve BarContainer.
ax.scatter	`ax.scatter(df['unidades'],df['total'],alpha=0.7)`	Dibuja pares de valores.	Para explorar relaciones.	**Matplotlib** Modifica el gráfico; devuelve PathCollection.
ax.hist	`ax.hist(df['total'],bins=5,edgecolor='white')`	Agrupa valores en intervalos.	Para ver la distribución.	**Matplotlib** Modifica el gráfico; devuelve tupla.
ax.boxplot	`ax.boxplot([a[:6],a[6:]],tick_labels=['Inicio','Final'])`	Dibuja cajas y extremos. **Nota:** Los extremos no son errores por definición.	Para comparar distribuciones.	**Matplotlib** Modifica el gráfico; devuelve dict.
ax.imshow	`ax.imshow(corr,vmin=-1,vmax=1,cmap='coolwarm')`	Representa una matriz con colores.	Para visualizar correlaciones.	**Matplotlib** Modifica el gráfico; devuelve AxesImage.
ax.errorbar	`ax.errorbar([1,2,3],[20,30,40],yerr=[2,3,2],fmt='o')`	Añade barras de error.	Para mostrar incertidumbre conocida.	**Matplotlib** Modifica el gráfico; devuelve ErrorbarContainer.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
ax.fill_between	`ax.fill_between([1,2,3],[18,27,38],[22,33,42],alpha=0.2)`	Colorea una banda entre curvas.	Para mostrar un intervalo.	**Matplotlib** Modifica el gráfico; devuelve FillBetweenPolyCollection.
ax.axhline / ax.axvline	`ax.axhline(50,color='red',linestyle='--')`	Dibujan referencias horizontales o verticales.	Para marcar objetivos o eventos.	**Matplotlib** Modifica el gráfico; devuelve Line2D.

Matplotlib · Etiquetar y exportar

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
ax.set	`ax.set(title='Ventas',xlabel='Fecha',ylabel='Euros')`	Asigna títulos y etiquetas.	Para explicar el gráfico.	**Matplotlib** Modifica el gráfico; devuelve lista.
ax.set_xlim / ax.set_ylim	`ax.set_ylim(0,150)`	Fijan los límites de los ejes.	Para comparar escalas.	**Matplotlib** Modifica el gráfico; devuelve tupla.
ax.set_yscale	`ax.set_yscale('log')`	Cambia la escala vertical. **Nota:** Ceros y negativos no caben en escala logarítmica.	Para valores de órdenes distintos.	**Matplotlib** Modifica el gráfico; devuelve None.
ax.legend	`ax.plot([1,2],label='Serie'); ax.legend(loc='best')`	Muestra etiquetas de las series.	Para distinguir líneas.	**Matplotlib** Modifica el gráfico; devuelve Legend.
ax.grid	`ax.grid(axis='y',alpha=0.3)`	Añade una cuadrícula.	Para facilitar comparaciones.	**Matplotlib** Modifica el gráfico; devuelve None.
ax.tick_params	`ax.tick_params(axis='x',labelrotation=45)`	Ajusta marcas y etiquetas.	Para evitar solapamientos.	**Matplotlib** Modifica el gráfico; devuelve None.
ax.annotate	`ax.annotate('Objetivo',xy=(2,50),xytext=(3,70),arrowprops={'arrowstyle':'->'})`	Coloca una anotación con flecha.	Para destacar un punto.	**Matplotlib** Modifica el gráfico; devuelve Annotation.
fig.autofmt_xdate	`fig.autofmt_xdate(rotation=30)`	Distribuye etiquetas de fechas.	Para un calendario legible.	**Matplotlib** Modifica el gráfico; devuelve None.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
fig.colorbar	`im=ax.imshow(corr); fig.colorbar(im,ax=ax)`	Añade una escala de color.	Para interpretar una matriz.	**Matplotlib** Modifica el gráfico; devuelve Colorbar.
fig.savefig	`fig.savefig('grafico.png',dpi=200,bbox_inches ='tight')`	Guarda la figura en un archivo. **Nota:** Exporta antes de cerrar la figura.	Para incluirla en un informe.	**Matplotlib** Modifica el gráfico; devuelve None.
plt.show / plt.close	`plt.close(fig)`	Muestran o cierran figuras. **Nota:** plt.show() requiere una interfaz para abrir una ventana.	Para revisar o liberar recursos.	**Matplotlib** Modifica el gráfico; devuelve None.

Seaborn · preparación

Exploración estadística directamente desde un DataFrame. Versión probada: 0.13.2.

Importaciones

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Objetos de las fichas

```
fig, ax = plt.subplots(figsize=(7,4))
corr = df[['unidades','precio','total']].corr()
```

sns es Seaborn; ax es un eje de Matplotlib. Seaborn utiliza Matplotlib para dibujar.

scatterplot, histplot y boxplot admiten ax. relplot, catplot y displot crean figuras propias.

La agregación predeterminada de lineplot y barplot puede cambiar lo que ves: revisa estimator y errorbar.

Seaborn · Explorar un DataFrame

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
sns.set_theme	`sns.set_theme(style='whitegrid',context='notebook')`	Configura el estilo gráfico.	Para una apariencia coherente.	**Seaborn** Modifica el gráfico; devuelve None.
sns.scatterplot	`sns.scatterplot(data=df,x='unidades',y='total',hue='ciudad',ax=ax)`	Dibuja dispersión por categorías.	Para comparar relaciones por ciudad.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.lineplot	`sns.lineplot(data=df,x='fecha',y='total',errorbar=None,ax=ax)`	Dibuja tendencias y agrega repeticiones. **Nota:** Por defecto estima la media para valores repetidos de x.	Para series temporales.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.barplot	`sns.barplot(data=df,x='cliente',y='total',estimator='sum',errorbar=None,ax=ax)`	Dibuja una estadística por categoría. **Nota:** El estimador predeterminado es la media, no la suma.	Para comparar facturación total.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.countplot	`sns.countplot(data=df,x='ciudad',ax=ax)`	Cuenta registros por categoría.	Para comparar número de ventas.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.histplot	`sns.histplot(data=df,x='total',bins=5,ax=ax)`	Dibuja un histograma.	Para explorar importes.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.kdeplot	`sns.kdeplot(data=df,x='total',fill=True,ax=ax)`	Estima una densidad suave. **Nota:** Requiere datos suficientes y no constantes.	Para explorar una distribución.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.ecdfplot	`sns.ecdfplot(data=df,x='total',ax=ax)`	Dibuja la proporción acumulada.	Para leer percentiles visualmente.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
sns.boxplot	`sns.boxplot(data=df,x='ciudad',y='total',ax=ax)`	Compara medianas y dispersión. **Nota:** Seaborn 0.13.2 emite un aviso interno de vert con Matplotlib 3.11; la llamada funciona. Usa ax.boxplot si necesitas evitarlo.	Para contrastar ciudades.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.violinplot	`sns.violinplot(data=df,x='ciudad',y='total',cut=0,ax=ax)`	Combina densidad y distribución.	Para comparar formas con suficientes datos.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.stripplot	`sns.stripplot(data=df,x='ciudad',y='total',jitter=True,ax=ax)`	Muestra observaciones individuales.	Para ver los registros de cada grupo.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.heatmap	`sns.heatmap(corr,annot=True,vmin=-1,vmax=1,cmap='coolwarm',ax=ax)`	Colorea una matriz con valores.	Para consultar correlaciones.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.regplot	`sns.regplot(data=df,x='unidades',y='total',ci=None,ax=ax)`	Añade un ajuste lineal a la dispersión. **Nota:** La asociación no demuestra causalidad.	Para explorar una tendencia.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.
sns.residplot	`sns.residplot(data=df,x='unidades',y='total',ax=ax)`	Dibuja residuos de un ajuste.	Para buscar patrones no explicados.	**Seaborn** Modifica el gráfico; devuelve eje gráfico.

Seaborn · Crear paneles y ajustar el estilo

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
sns.relplot	`sns.relplot(data=df,x='unidades',y='total', col='ciudad',kind='scatter')`	Crea paneles de relaciones. **Nota:** Crea su propia figura; no acepta ax.	Para comparar ciudades lado a lado.	**Seaborn** Modifica el gráfico; devuelve paneles.
sns.catplot	`sns.catplot(data=df,x='cliente',y='total',col ='ciudad',kind='bar',errorbar=None)`	Crea paneles categóricos.	Para comparar grupos en ciudades.	**Seaborn** Modifica el gráfico; devuelve paneles.
sns.displot	`sns.displot(data=df,x='total',col='ciudad', bins=4)`	Crea paneles de distribución.	Para comparar histogramas.	**Seaborn** Modifica el gráfico; devuelve paneles.
sns.pairplot	`sns.pairplot(df,vars=['unidades','precio', 'total'],hue='ciudad',diag_kind='hist')`	Cruza pares de variables numéricas.	Para exploración multivariable.	**Seaborn** Modifica el gráfico; devuelve paneles.
sns.jointplot	`sns.jointplot(data=df,x='unidades',y='total', kind='scatter')`	Combina relación y distribuciones marginales.	Para estudiar dos variables.	**Seaborn** Modifica el gráfico; devuelve paneles.
sns.color_palette	`sns.color_palette('colorblind',n_colors=3)`	Devuelve una paleta de colores.	Para distinguir grupos.	**Seaborn** Modifica el gráfico; devuelve _ColorPalette.
sns.despine	`sns.despine(ax=ax)`	Retira bordes del gráfico.	Para limpiar su presentación.	**Seaborn** Modifica el gráfico; devuelve None.

SciPy · preparación

Estadística inferencial, interpolación y optimización. Versión probada: 1.18.1.

Importaciones

```
from scipy import stats, interpolate
from scipy import optimize, integrate, signal
from scipy import sparse as sp
```

Objetos de las fichas

```
a = df['total'].to_numpy(dtype=float)
Xn = df[['unidades','precio']].to_numpy()
```

stats, interpolate y optimize son módulos importados explícitamente.

Los contrastes son recetas de sintaxis: los datos didácticos no permiten conclusiones de negocio.

Revisa independencia, diseño y tamaño del efecto. Una p pequeña no demuestra causalidad.

SciPy · Describir y contrastar muestras

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
stats.describe	`stats.describe(a,ddof=1)`	Resume tamaño, extremos y momentos.	Para descripción numérica compacta.	**SciPy** Devuelve DescribeResult.
stats.zscore	`stats.zscore(a,ddof=1)`	Estandariza por media y desviación. **Nota:** En modelos ajusta la escala solo con entrenamiento.	Para comparar escalas.	**SciPy** Devuelve array NumPy.
stats.iqr	`stats.iqr(a)`	Calcula rango intercuartílico.	Para dispersión robusta.	**SciPy** Devuelve número.
stats.sem	`stats.sem(a,ddof=1)`	Calcula error estándar de la media.	Para precisión de una media.	**SciPy** Devuelve número.
stats.trim_mean	`stats.trim_mean(a,proportiontocut=0.1)`	Calcula una media recortada.	Para reducir influencia de extremos.	**SciPy** Devuelve número.
stats.pearsonr	`stats.pearsonr(df['unidades'],df['total'])`	Calcula correlación lineal y p-valor. **Nota:** Devuelve statistic y pvalue; revisa los supuestos.	Para estudiar asociación lineal.	**SciPy** Devuelve PearsonRResult.
stats.spearmanr	`stats.spearmanr(df['unidades'],df['total'])`	Calcula correlación por rangos.	Para asociaciones monótonas.	**SciPy** Devuelve SignificanceResult.
stats.ttest_ind	`stats.ttest_ind(a[:6],a[6:],equal_var=False)`	Compara medias de muestras independientes. **Nota:** Revisa independencia; el p-valor no mide tamaño del efecto.	Para una prueba de Welch.	**SciPy** Devuelve TtestResult.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
stats.ttest_rel	`stats.ttest_rel(a,a+np.linspace(0,2,len(a)))`	Compara medidas emparejadas.	Para antes y después de los mismos sujetos.	**SciPy** Devuelve TtestResult.
stats.mannwhitneyu	`stats.mannwhitneyu(a[:6],a[6:],alternative= 'two-sided')`	Compara muestras por rangos. **Nota:** No es siempre una prueba de medianas.	Para muestras independientes.	**SciPy** Devuelve MannwhitneyuResult.
stats.wilcoxon	`stats.wilcoxon(a,a+np.arange(1,len(a)+1))`	Compara diferencias emparejadas por rangos.	Para medidas repetidas.	**SciPy** Devuelve WilcoxonResult.
stats.f_oneway	`stats.f_oneway(a[:4],a[4:8],a[8:])`	Realiza ANOVA de un factor. **Nota:** Revisa independencia, residuos y varianzas.	Para comparar medias de tres grupos.	**SciPy** Devuelve F_onewayResult.
stats.kruskal	`stats.kruskal(a[:4],a[4:8],a[8:])`	Compara varios grupos por rangos.	Para contrastar distribuciones.	**SciPy** Devuelve KruskalResult.
stats.chi2_contingency	`stats.chi2_contingency([[20,15],[12,25]])`	Contrasta independencia entre categorías. **Nota:** Revisa frecuencias esperadas; no pases porcentajes.	Para tablas de frecuencias.	**SciPy** Devuelve Chi2ContingencyResult.
stats.shapiro	`stats.shapiro(a)`	Contrasta normalidad. **Nota:** No decide por sí solo qué análisis es válido.	Para diagnóstico de distribución.	**SciPy** Devuelve ShapiroResult.
stats.levene	`stats.levene(a[:6],a[6:],center='median')`	Contrasta igualdad de varianzas.	Para revisar dispersión entre grupos.	**SciPy** Devuelve LeveneResult.
stats.linregress	`stats.linregress(df['unidades'],df['total'])`	Ajusta una recta y devuelve diagnósticos. **Nota:** Incluye slope, intercept y rvalue.	Para regresión simple.	**SciPy** Devuelve LinregressResult.

SciPy · Distribuciones y cálculo científico

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
stats.norm.cdf / ppf	`stats.norm.ppf(0.975)`	Calculan probabilidades o cuantiles normales.	Para puntos críticos teóricos.	**SciPy** Devuelve número.
stats.norm.rvs	`stats.norm.rvs(size=5,random_state=42)`	Genera una muestra normal.	Para simulaciones reproducibles.	**SciPy** Devuelve array NumPy.
stats.bootstrap	`stats.bootstrap((a,),np.mean,n_resamples=999, rng=np.random.default_rng(42))`	Estima un intervalo por remuestreo. **Nota:** La muestra debe representar el proceso analizado.	Para incertidumbre de un estadístico.	**SciPy** Devuelve BootstrapResult.
interpolate.PchipInterpolator	`interpolate.PchipInterpolator([0,1,2],[10,20, 15])([0.5,1.5])`	Interpola conservando la forma local. **Nota:** Ordena x y evita duplicados.	Para estimar valores intermedios.	**SciPy** Devuelve array NumPy.
optimize.curve_fit	`optimize.curve_fit(lambda x,m,b:m*x+b,df[ 'unidades'],a)`	Ajusta parámetros de una función. **Nota:** Devuelve parámetros y su covarianza.	Para relaciones paramétricas.	**SciPy** Devuelve tupla.
optimize.minimize	`optimize.minimize(lambda z:(z[0]-3)**2,x0=[0. ])`	Busca un mínimo numérico. **Nota:** Comprueba success y message.	Para optimizar un coste.	**SciPy** Devuelve OptimizeResult.
integrate.trapezoid	`integrate.trapezoid(a,x=np.arange(len(a)))`	Integra con la regla del trapecio.	Para el área bajo una curva.	**SciPy** Devuelve número.
signal.find_peaks	`signal.find_peaks(a,prominence=5)`	Localiza máximos locales.	Para detectar picos.	**SciPy** Devuelve tupla.
sp.csr_array	`sp.csr_array(Xn)`	Construye una matriz dispersa CSR.	Para muchas variables con ceros.	**SciPy** Devuelve csr_array.

scikit-learn · preparación

Preparación de variables, modelos y evaluación. Versión probada: 1.9.1.

Importaciones

```
from sklearn.model_selection import (
    train_test_split, cross_val_score,
    GridSearchCV, TimeSeriesSplit)
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler,
    OneHotEncoder, OrdinalEncoder,
    PolynomialFeatures)
from sklearn.impute import SimpleImputer
from sklearn.compose import (
    ColumnTransformer, make_column_selector)
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.linear_model import (
    LinearRegression, Ridge, Lasso,
    LogisticRegression)
from sklearn.ensemble import (
    RandomForestRegressor, RandomForestClassifier,
    HistGradientBoostingRegressor)
from sklearn.dummy import DummyRegressor, DummyClassifier
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.inspection import permutation_importance
from sklearn.metrics import (
    mean_absolute_error, root_mean_squared_error,
    r2_score, accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix,
    classification_report, roc_auc_score)
```

Objetos de las fichas

```
X = df[['unidades','precio']]
y = df['total']
yc = (y > y.median()).astype(int)
X_train, X_test, y_train, y_test = (
    train_test_split(X,y,test_size=0.25,
                     random_state=42))
scaler = StandardScaler().fit(X_train)
modelo = LinearRegression().fit(X_train,y_train)
pred = modelo.predict(X_test)
clasificador = LogisticRegression(
    max_iter=1000).fit(X,yc)
cl_pred = clasificador.predict(X)
bosque = RandomForestRegressor(
    n_estimators=50,random_state=42
).fit(X_train,y_train)
```

X contiene variables; y es el objetivo numérico; yc es la clase binaria. Los sufijos train/test indican entrenamiento/prueba.

Los nombres modelo, scaler, clasificador y bosque son variables; puedes cambiarlos.

Las métricas de clasificación de estas fichas usan entrenamiento solo para mostrar sintaxis. En un proyecto, evalúa con datos reservados.

fit aprende y modifica el estimador. predict y transform usan lo aprendido. Los atributos terminados en _ suelen existir después de fit.

scikit-learn · Preparar variables

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
train_test_split	`train_test_split(X,y,test_size=0.25, random_state=42)`	Separa entrenamiento y prueba. **Nota:** En series temporales divide por fecha.	Para evaluar datos no vistos.	**scikit-learn** Devuelve lista.
SimpleImputer	`SimpleImputer(strategy='median').fit_transform(X)`	Rellena nulos aprendiendo una regla. **Nota:** Ajusta únicamente con entrenamiento en un flujo real.	Para preparar variables numéricas.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
StandardScaler	`StandardScaler().fit_transform(X)`	Centra y escala variables. **Nota:** Los árboles no suelen necesitarlo.	Para modelos sensibles a escalas.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
MinMaxScaler	`MinMaxScaler().fit_transform(X)`	Escala a un intervalo.	Para normalizar rangos.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
RobustScaler	`RobustScaler().fit_transform(X)`	Escala por mediana y cuantiles.	Para variables con extremos.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
OneHotEncoder	`OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit_transform(df[['ciudad']])`	Convierte categorías en columnas binarias. **Nota:** Con muchas categorías conserva salida dispersa.	Para incluir ciudades en un modelo.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
OrdinalEncoder	`OrdinalEncoder(handle_unknown='use_encoded_value',unknown_value=-1).fit_transform(df[['ciudad']])`	Codifica categorías como números. **Nota:** Los códigos no crean un orden real en categorías nominales.	Para un orden definido o modelos adecuados.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
PolynomialFeatures	`PolynomialFeatures(degree=2,include_bias=False).fit_transform(X)`	Crea potencias e interacciones.	Para relaciones no lineales sencillas.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
ColumnTransformer	`ColumnTransformer([('num',StandardScaler(),['unidades','precio']),('cat',OneHotEncoder(handle_unknown='ignore'),['ciudad']),])`	Aplica reglas distintas por columna.	Para combinar números y categorías.	**scikit-learn** Devuelve ColumnTransformer.
make_column_selector	`make_column_selector(dtype_include=np.number)(df)`	Selecciona columnas por tipo.	Para esquemas con muchas variables.	**scikit-learn** Devuelve lista.
make_pipeline	`make_pipeline(StandardScaler(),Ridge(alpha=1))`	Encadena preparación y modelo.	Para automatizar un flujo de aprendizaje.	**scikit-learn** Devuelve Pipeline.
Pipeline	`Pipeline([('escala',StandardScaler()),('modelo',Ridge())])`	Encadena pasos con nombres explícitos.	Para ajustar parámetros de cada etapa.	**scikit-learn** Devuelve Pipeline.
set_output	`StandardScaler().set_output(transform='pandas').fit_transform(X)`	Conserva nombres en una salida DataFrame. **Nota:** No todos los transformadores admiten todas las salidas.	Para inspeccionar variables transformadas.	**scikit-learn** Aprende sobre el estimador; devuelve DataFrame.
get_feature_names_out	`scaler.get_feature_names_out()`	Devuelve nombres de variables de salida.	Para recuperar columnas tras transformar.	**scikit-learn** Devuelve array NumPy.
fit / transform	`scaler.fit(X_train); scaler.transform(X_test)`	Aprende con entrenamiento y aplica a prueba. **Nota:** fit modifica el estimador; transform devuelve datos.	Para evitar fuga de información.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.

scikit-learn · Ajustar y utilizar modelos

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
LinearRegression	`LinearRegression().fit(X_train,y_train)`	Ajusta una regresión lineal.	Para una referencia de predicción numérica.	**scikit-learn** Aprende sobre el estimador; devuelve LinearRegression.
Ridge	`Ridge(alpha=1).fit(X_train,y_train)`	Ajusta regresión con penalización L2.	Para coeficientes más estables.	**scikit-learn** Aprende sobre el estimador; devuelve Ridge.
Lasso	`Lasso(alpha=0.1,max_iter=10000).fit(X_train, y_train)`	Ajusta regresión con penalización L1. **Nota:** Escala las variables antes de comparar penalizaciones.	Para reducir coeficientes a cero.	**scikit-learn** Aprende sobre el estimador; devuelve Lasso.
RandomForestRegressor	`RandomForestRegressor(n_estimators=50, random_state=42).fit(X_train,y_train)`	Combina árboles de regresión.	Para relaciones no lineales.	**scikit-learn** Aprende sobre el estimador; devuelve RandomForestRegressor.
HistGradientBoostingRegressor	`HistGradientBoostingRegressor(max_iter=30, random_state=42).fit(X_train,y_train)`	Ajusta árboles mediante boosting. **Nota:** Estos datos son demasiado pequeños para juzgar rendimiento.	Para regresión tabular.	**scikit-learn** Aprende sobre el estimador; devuelve HistGradientBoostingRegressor.
LogisticRegression	`LogisticRegression(max_iter=1000).fit(X,yc)`	Ajusta un clasificador logístico.	Para predecir una categoría.	**scikit-learn** Aprende sobre el estimador; devuelve LogisticRegression.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
RandomForestClassifier	`RandomForestClassifier(n_estimators=50, random_state=42).fit(X,yc)`	Combina árboles de clasificación.	Para clases con relaciones no lineales.	**scikit-learn** Aprende sobre el estimador; devuelve RandomForestClassifier.
DummyRegressor / DummyClassifier	`DummyRegressor(strategy='mean').fit(X_train, y_train)`	Crean predicciones de referencia. **Nota:** En clasificación, usa DummyClassifier(strategy='most_frequent').	Para comprobar si un modelo aporta valor.	**scikit-learn** Aprende sobre el estimador; devuelve DummyRegressor.
KMeans	`KMeans(n_clusters=3,n_init=10,random_state=42).fit_predict(X)`	Asigna observaciones a grupos. **Nota:** Escala variables; un grupo no equivale a una clase real.	Para segmentar sin etiquetas.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
PCA	`PCA(n_components=1).fit_transform(StandardScaler().fit_transform(X))`	Reduce a componentes lineales.	Para resumir dimensiones numéricas.	**scikit-learn** Aprende sobre el estimador; devuelve array NumPy.
fit	`modelo.fit(X_train,y_train)`	Aprende parámetros a partir de datos. **Nota:** Modifica el estimador y suele devolverlo.	Para entrenar un estimador.	**scikit-learn** Aprende sobre el estimador; devuelve LinearRegression.
predict	`modelo.predict(X_test)`	Devuelve predicciones.	Para aplicar un modelo entrenado.	**scikit-learn** Devuelve array NumPy.
predict_proba	`clasificador.predict_proba(X)`	Devuelve probabilidades por clase. **Nota:** Consulta classes_ para el orden de las columnas.	Para ajustar un umbral.	**scikit-learn** Devuelve array NumPy.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
coef_	`modelo.coef_`	Devuelve coeficientes aprendidos. **Nota:** Solo tras fit y en modelos que los exponen.	Para revisar una regresión lineal.	**scikit-learn** Devuelve array NumPy.
feature_importances_	`bosque.feature_importances_`	Devuelve importancias basadas en árboles. **Nota:** Puede favorecer variables con muchas divisiones posibles.	Para una revisión inicial de variables.	**scikit-learn** Devuelve array NumPy.

scikit-learn · Evaluar y seleccionar modelos

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
mean_absolute_error	`mean_absolute_error(y_test,pred)`	Calcula error absoluto medio.	Para errores en unidades del objetivo.	**scikit-learn** Devuelve número.
root_mean_squared_error	`root_mean_squared_error(y_test,pred)`	Calcula raíz del error cuadrático medio.	Para penalizar errores grandes.	**scikit-learn** Devuelve número.
r2_score	`r2_score(y_test,pred)`	Compara con predecir la media. **Nota:** Puede ser negativo; no es porcentaje de aciertos.	Para evaluar regresión.	**scikit-learn** Devuelve número.
accuracy_score	`accuracy_score(yc,cl_pred)`	Calcula proporción de aciertos.	Para clases equilibradas.	**scikit-learn** Devuelve número.
precision_score / recall_score	`precision_score(yc,cl_pred,zero_division=0)`	Miden precisión o sensibilidad. **Nota:** precision: positivos predichos correctos; recall: positivos reales detectados.	Para revisar falsas alarmas y casos perdidos.	**scikit-learn** Devuelve número.
f1_score	`f1_score(yc,cl_pred,zero_division=0)`	Combina precisión y sensibilidad.	Para comparar clasificación binaria.	**scikit-learn** Devuelve número.
confusion_matrix	`confusion_matrix(yc,cl_pred)`	Cuenta aciertos y errores por clase.	Para ver clases confundidas.	**scikit-learn** Devuelve array NumPy.
classification_report	`classification_report(yc,cl_pred, zero_division=0)`	Resume métricas por clase.	Para diagnóstico de clasificación.	**scikit-learn** Devuelve texto.
roc_auc_score	`roc_auc_score(yc,clasificador.predict_proba(X)[:,1])`	Evalúa la ordenación de positivos. **Nota:** Usa probabilidades o puntuaciones, no etiquetas finales.	Para comparar puntuaciones.	**scikit-learn** Devuelve número.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
cross_val_score	`cross_val_score(make_pipeline(StandardScaler(),Ridge()),X,y,cv=3,scoring='neg_mean_absolute_error')`	Evalúa varios repartos de datos. **Nota:** Las métricas de error llevan signo negativo.	Para estimar variación de rendimiento.	**scikit-learn** Devuelve array NumPy.
GridSearchCV	`GridSearchCV(make_pipeline(StandardScaler(),Ridge()),{'ridge__alpha':[0.1,1,10]},cv=3).fit(X,y)`	Busca parámetros mediante validación cruzada. **Nota:** Reserva una prueba final independiente.	Para comparar ajustes de un pipeline.	**scikit-learn** Aprende sobre el estimador; devuelve GridSearchCV.
TimeSeriesSplit	`list(TimeSeriesSplit(n_splits=3).split(X))`	Genera particiones temporales crecientes. **Nota:** Ordena los datos por fecha.	Para validar sin ver el futuro.	**scikit-learn** Devuelve lista.
permutation_importance	`permutation_importance(modelo,X_test,y_test,n_repeats=5,random_state=42)`	Mide pérdida al permutar variables. **Nota:** Las variables correlacionadas pueden ocultar su importancia.	Para importancia sobre datos reservados.	**scikit-learn** Devuelve Bunch.

PyArrow · preparación

Intercambio columnar, Parquet y lectura por lotes. Versión probada: 26.0.0.

Importaciones

```
import pyarrow as pa
import pyarrow.compute as pc
import pyarrow.parquet as pq
import pyarrow.dataset as ds
```

Objetos de las fichas

```
tabla = pa.Table.from_pandas(
    df, preserve_index=False)
pq.write_table(tabla,'ventas.parquet',
               compression='zstd')
pf = pq.ParquetFile('ventas.parquet')
dataset = ds.dataset('ventas.parquet',
                     format='parquet')
```

pa = Arrow; pc = cálculo; pq = Parquet; ds = datasets.

tabla es una Table; pf es ParquetFile; dataset es un Dataset.

to_pandas y to_table materializan datos en memoria. Para archivos grandes procesa lotes con iter_batches.

PyArrow · Intercambiar tablas y tipos

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
pa.Table.from_pandas	`pa.Table.from_pandas(df,preserve_index=False)`	Convierte pandas en una tabla Arrow.	Para intercambio columnar.	PyArrow Devuelve Table.
tabla.to_pandas	`tabla.to_pandas()`	Convierte una tabla Arrow a pandas.	Para volver al análisis en pandas.	PyArrow Devuelve DataFrame.
pa.array	`pa.array([10,None,30],type=pa.int64())`	Crea un array con tipo explícito.	Para enteros con nulos.	PyArrow Devuelve Int64Array.
pa.schema	`pa.schema([('cliente',pa.string()),('total', pa.float64())])`	Define nombres y tipos.	Para validar un intercambio.	PyArrow Devuelve Schema.
tabla.schema	`tabla.schema`	Devuelve el esquema.	Para revisar tipos.	PyArrow Devuelve Schema.
tabla.num_rows	`tabla.num_rows`	Devuelve el número de filas.	Para comprobar un lote.	PyArrow Devuelve entero.
tabla.select	`tabla.select(['cliente','total'])`	Selecciona columnas.	Para reducir datos antes de convertir.	PyArrow Devuelve Table.
tabla.slice	`tabla.slice(0,3)`	Obtiene un tramo de filas.	Para inspeccionar un lote.	PyArrow Devuelve Table.
tabla.filter	`tabla.filter(pc.greater(tabla['total'],40))`	Filtra con una máscara Arrow.	Para reducir datos sin usar pandas.	PyArrow Devuelve Table.
tabla.sort_by	`tabla.sort_by([('total','descending')])`	Ordena por columnas.	Para preparar una clasificación.	PyArrow Devuelve Table.
pa.concat_tables	`pa.concat_tables([tabla,tabla.slice(0,2)])`	Concatena tablas compatibles.	Para unir lotes del mismo esquema.	PyArrow Devuelve Table.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
pc.cast	`pc.cast(tabla['unidades'],pa.float64())`	Convierte el tipo de una columna.	Para adaptar un esquema.	**PyArrow** Devuelve ChunkedArray.
pc.fill_null	`pc.fill_null(pa.array([1,None]),0)`	Rellena valores nulos.	Para preparar una columna.	**PyArrow** Devuelve Int64Array.
pc.sum	`pc.sum(tabla['total'])`	Suma una columna Arrow.	Para resumir sin convertir a pandas.	**PyArrow** Devuelve DoubleScalar.

PyArrow · Leer Parquet y procesar conjuntos

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
pq.write_table	`pq.write_table(tabla,'ventas.parquet', compression='zstd')`	Escribe Parquet.	Para comprimir y conservar tipos.	**PyArrow** Escribe archivos; devuelve None.
pq.read_table	`pq.read_table('ventas.parquet',columns=['cliente','total'])`	Lee columnas de un Parquet.	Para cargar solo lo necesario.	**PyArrow** Devuelve Table.
pq.read_schema	`pq.read_schema('ventas.parquet')`	Lee el esquema de un Parquet.	Para inspeccionar tipos sin cargar datos.	**PyArrow** Devuelve Schema.
pq.ParquetFile	`pq.ParquetFile('ventas.parquet')`	Abre metadatos y grupos de filas.	Para planificar una lectura.	**PyArrow** Devuelve ParquetFile.
pf.metadata	`pf.metadata`	Devuelve metadatos del archivo.	Para conocer filas y grupos.	**PyArrow** Devuelve FileMetaData.
pf.iter_batches	`list(pf.iter_batches(batch_size=4,columns=['total']))`	Lee lotes de registros. **Nota:** Procesa el iterador directamente; list es solo para este ejemplo pequeño.	Para limitar memoria por lote.	**PyArrow** Devuelve lista.
ds.dataset	`ds.dataset('ventas.parquet',format='parquet')`	Describe un conjunto de archivos.	Para explorar sin cargarlos aún.	**PyArrow** Devuelve FileSystemDataset.
ds.field	`ds.field('total')>40`	Construye una expresión de filtro.	Para filtrar al leer.	**PyArrow** Devuelve Expression.
dataset.to_table	`dataset.to_table(columns=['cliente','total'], filter=ds.field('total')>40)`	Lee columnas y filas seleccionadas.	Para materializar un subconjunto.	**PyArrow** Devuelve Table.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
ds.write_dataset	`ds.write_dataset(tabla,'dataset_salida', format='parquet',partitioning=['ciudad'], existing_data_behavior= 'overwrite_or_ignore')`	Escribe archivos particionados. **Nota:** La política elegida puede sobrescribir archivos del mismo nombre.	Para organizar datos por categoría.	**PyArrow** Escribe archivos; devuelve None.

DuckDB · preparación

SQL analítico sobre pandas y archivos locales. Versión probada: 1.5.6.

Importaciones

```
import duckdb
```

Objetos de las fichas

```
con = duckdb.connect(':memory:')
con.register('ventas',df)
con.register('clientes',clientes)
rel = con.from_df(df)
df.to_csv('ventas.csv',index=False)
df.to_parquet('ventas.parquet',index=False)
```

con es una conexión; rel es una relación con filas y columnas.

Registra ventas y clientes para consultar estas tablas por nombre SQL.

Muchos pasos relacionales son diferidos; df() ejecuta y materializa el resultado. Cierra con al terminar.

DuckDB · Consultar pandas mediante SQL

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
duckdb.connect	`duckdb.connect(':memory:')`	Abre una base analítica en memoria.	Para SQL local sin servidor.	**DuckDB** Devuelve DuckDBPyConnection.
con.register	`con.register('ventas',df)`	Expone un DataFrame como vista SQL. **Nota:** No equivale a copiar una tabla persistente.	Para consultar pandas con SQL.	**DuckDB** Cambia el estado indicado; devuelve DuckDBPyConnection.
con.sql	`con.sql(( 'SELECT cliente, SUM(total) AS tot' 'al FROM ventas GROUP BY cliente' ))`	Construye una consulta SQL.	Para agrupar con SQL.	**DuckDB** Devuelve DuckDBPyRelation.
rel.df	`rel.df()`	Materializa como DataFrame. **Nota:** El resultado completo debe caber en memoria.	Para continuar con pandas.	**DuckDB** Devuelve DataFrame.
con.execute	`con.execute(( 'SELECT * FROM ventas WHERE total ' '> ?' ), [40]).df()`	Ejecuta SQL con parámetros. **Nota:** Los parámetros sustituyen valores, no nombres de tablas.	Para filtrar por valores variables.	**DuckDB** Devuelve DataFrame.
con.from_df	`con.from_df(df)`	Crea una relación desde pandas.	Para usar la API relacional.	**DuckDB** Devuelve DuckDBPyRelation.
rel.filter	`rel.filter('total > 40')`	Filtra filas de una relación.	Para reducir datos antes de exportar.	**DuckDB** Devuelve DuckDBPyRelation.
rel.project	`rel.project('cliente, total*1.21 AS con_iva')`	Selecciona o calcula columnas.	Para transformar con expresiones SQL.	**DuckDB** Devuelve DuckDBPyRelation.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
rel.aggregate	`rel.aggregate('cliente, sum(total) AS total', 'cliente')`	Agrupa y resume una relación.	Para totales por cliente.	**DuckDB** Devuelve DuckDBPyRelation.
rel.order	`rel.order('total DESC')`	Ordena una relación.	Para clasificar resultados.	**DuckDB** Devuelve DuckDBPyRelation.
rel.limit	`rel.limit(3)`	Limita filas.	Para inspeccionar resultados.	**DuckDB** Devuelve DuckDBPyRelation.
rel.columns / rel.types	`rel.columns`	Devuelven nombres o tipos SQL.	Para revisar un resultado.	**DuckDB** Devuelve lista.
con.unregister	`con.unregister('ventas')`	Retira una vista registrada.	Para liberar una referencia al DataFrame.	**DuckDB** Cambia el estado indicado; devuelve DuckDBPyConnection.
con.close	`con.close()`	Cierra la conexión.	Para terminar una sesión.	**DuckDB** Cambia el estado indicado; devuelve None.

DuckDB · Leer archivos y guardar resultados

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
con.read_csv	`con.read_csv('ventas.csv',header=True)`	Consulta un CSV como relación.	Para filtrar antes de convertir a pandas.	**DuckDB** Devuelve DuckDBPyRelation.
con.read_parquet	`con.read_parquet('ventas.parquet')`	Consulta un Parquet como relación.	Para usar un formato columnar.	**DuckDB** Devuelve DuckDBPyRelation.
rel.write_parquet	`rel.write_parquet('consulta.parquet', compression='zstd')`	Exporta una relación a Parquet.	Para evitar un DataFrame intermedio.	**DuckDB** Escribe archivos; devuelve None.
rel.write_csv	`rel.write_csv('consulta.csv',header=True)`	Exporta una relación a CSV.	Para intercambiar resultados.	**DuckDB** Escribe archivos; devuelve None.
CREATE TABLE AS	`con.execute(( 'CREATE TABLE copia AS SELECT * FR' 'OM ventas' ))`	Copia resultados a una tabla física. **Nota:** Una conexión a archivo permite persistir entre sesiones.	Para fijar una instantánea.	**DuckDB** Cambia el estado indicado; devuelve DuckDBPyConnection.
JOIN	`con.sql(( 'SELECT v.*, c.segmento FROM venta' 's v LEFT JOIN clientes c USING (c' 'liente)' )).df()`	Combina tablas por una clave. **Nota:** Claves repetidas multiplican filas.	Para enriquecer ventas con clientes.	**DuckDB** Devuelve DataFrame.
OVER	`con.sql(( 'SELECT fecha,total,sum(total) OVE' 'R (ORDER BY fecha) AS acumulado F' 'ROM ventas' )).df()`	Calcula ventanas SQL.	Para acumulados sin perder filas.	**DuckDB** Devuelve DataFrame.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado

EXPLAIN	`con.execute((`
	`  'EXPLAIN SELECT * FROM ventas WHER'`
	`  'E total > 40'`
	`)).fetchall()`
		Muestra el plan de consulta.	Para entender la ejecución.	**DuckDB**
				Devuelve lista.

SQLAlchemy · preparación

Conexiones y transacciones con bases SQL. Versión probada: 2.1.4.

Importaciones

```
from sqlalchemy import (
    create_engine, text, inspect, MetaData,
    Table, select, func)
```

Objetos de las fichas

```
engine = create_engine(
    'sqlite+pysqlite:///:memory:')
df.to_sql('ventas',engine,index=False)
ventas_sql = Table('ventas',MetaData(),
                   autoload_with=engine)
```

engine gestiona conexiones; conn es una conexión abierta; ventas_sql es una Table reflejada.

La preparación crea una SQLite en memoria para que no necesites un servidor. En la práctica cambia la URL y el controlador.

Usa parámetros para valores y engine.begin() para transacciones. No concatentes entradas del usuario en SQL.

SQLAlchemy · Conectar y consultar bases de datos

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
create_engine	`create_engine('sqlite+pysqlite:///:memory:')`	Configura un motor de conexión. **Nota:** Otros servidores necesitan controlador y URL adecuados.	Para conectar pandas a SQL.	**SQLAlchemy** Devuelve Engine.

engine.connect	`with engine.connect() as conn:`
	`  pd.read_sql_query(text(`
	`  'SELECT * FROM ventas'),conn)`
		Abre una conexión con cierre automático.	Para leer dentro de una sesión.	**SQLAlchemy** Abre y cierra conn; la lectura produce un DataFrame.

engine.begin	`with engine.begin() as conn:`
	`  conn.execute(text((`
	`  'UPDATE ventas SET total=total WHE'`
	`  'RE total<0'`
	`)))`
		Abre una transacción automática. **Nota:** Confirma al salir sin error y revierte si falla.	Para confirmar cambios juntos.	**SQLAlchemy** Transacción; confirma o revierte al salir.

text	`text((`
	`  'SELECT * FROM ventas WHERE total '`
	`  '> :minimo'`
	`))`
		Construye SQL parametrizable.	Para consultas con parámetros.	**SQLAlchemy** Devuelve TextClause.

pd.read_sql_query	`pd.read_sql_query(text((`
	`  'SELECT * FROM ventas WHERE total '`
	`  '> :m'`
	`)),engine,params={'m':40})`
		Lee una consulta como DataFrame.	Para traer un subconjunto de la base.	**SQLAlchemy** Devuelve DataFrame.

pd.read_sql_table	`pd.read_sql_table('ventas',engine,columns=[` `  'cliente','total'])`	Lee columnas de una tabla SQL.	Para cargar tablas con SQLAlchemy.	**SQLAlchemy** Devuelve DataFrame.
df.to_sql	`df.to_sql('ventas_salida',engine,index=False,` `  if_exists='append',chunksize=1000)`	Escribe filas de pandas en SQL. **Nota:** append no evita duplicados; replace elimina la tabla anterior.	Para cargar datos por lotes.	**SQLAlchemy** Cambia el estado indicado; devuelve entero.
inspect	`inspect(engine).get_table_names()`	Lista tablas.	Para explorar el esquema.	**SQLAlchemy** Devuelve lista.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
Inspector.get_columns	`inspect(engine).get_columns('ventas')`	Devuelve metadatos de columnas.	Para revisar tipos y nulabilidad.	**SQLAlchemy** Devuelve lista.
MetaData	`MetaData()`	Crea un contenedor de esquema.	Para SQLAlchemy Core.	**SQLAlchemy** Devuelve MetaData.
Table	`Table('ventas',MetaData(),autoload_with= engine)`	Refleja una tabla existente.	Para construir SQL con objetos.	**SQLAlchemy** Devuelve Table.
select	`select(ventas_sql.c.cliente,ventas_sql.c. total).where(ventas_sql.c.total>40)`	Construye una selección SQL.	Para consultas sin SQL textual.	**SQLAlchemy** Devuelve Select.
func.sum	`select(func.sum(ventas_sql.c.total))`	Construye una suma SQL.	Para resumir en la base.	**SQLAlchemy** Devuelve Select.
Connection.execute	`with engine.connect() as conn: resultado=conn.execute(select(ventas_sql. c.total)).all()`	Ejecuta una expresión SQL.	Para obtener filas sin pandas.	**SQLAlchemy** resultado contiene una lista de filas.
engine.dispose	`engine.dispose()`	Libera conexiones del pool. **Nota:** SQLite en memoria puede perder datos al cerrar la conexión física.	Para cerrar recursos del motor.	**SQLAlchemy** Cambia el estado indicado; devuelve None.

openpyxl · preparación

Edición de libros, celdas y formatos de Excel. Versión probada: 3.1.5.

Importaciones

```
from openpyxl import Workbook, load_workbook
from openpyxl.utils.dataframe import dataframe_to_rows
from openpyxl.styles import Font, PatternFill, Alignment
from openpyxl.formatting.rule import ColorScaleRule
from openpyxl.worksheet.table import Table as ExcelTable
from openpyxl.worksheet.datavalidation import DataValidation
from openpyxl.chart import BarChart, Reference
```

Objetos de las fichas

```
wb = Workbook()
ws = wb.active
ws.title = 'Ventas'
for fila in dataframe_to_rows(
        df,index=False,header=True):
    ws.append(fila)
wb.save('plantilla.xlsx')
```

wb es un Workbook, el libro; ws es una Worksheet, la hoja.

Usa pandas para transformar tablas y openpyxl para conservar o editar formatos y celdas.

Escribe fórmulas con nombres de función en inglés. openpyxl las guarda pero no las calcula. Cierra libros de lectura al terminar.

openpyxl · Abrir libros y editar celdas

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
Workbook	`Workbook()`	Crea un libro Excel vacío.	Para una entrega nueva.	**openpyxl** Devuelve Workbook.
load_workbook	`load_workbook('plantilla.xlsx')`	Abre un libro existente. **Nota:** data_only=True lee valores almacenados; no calcula fórmulas.	Para editar una plantilla.	**openpyxl** Devuelve Workbook.
wb.active	`wb.active`	Devuelve la hoja activa.	Para empezar con el libro.	**openpyxl** Devuelve Worksheet.
wb.sheetnames	`wb.sheetnames`	Lista nombres de hojas.	Para encontrar una hoja.	**openpyxl** Devuelve lista.
wb.create_sheet	`wb.create_sheet('Resumen')`	Añade una hoja.	Para separar tablas.	**openpyxl** Devuelve Worksheet.
wb[hoja]	`wb['Ventas']`	Accede a una hoja por nombre.	Para editar la hoja correcta.	**openpyxl** Devuelve Worksheet.
ws[cell].value	`ws['A1'].value`	Lee contenido de una celda.	Para revisar títulos o valores.	**openpyxl** Devuelve texto.
Asignación de celda	`ws['A1']='Cliente'`	Escribe en una celda.	Para cambiar un encabezado.	**openpyxl** Modifica el libro o la hoja.
ws.cell	`ws.cell(row=2,column=2,value=99)`	Accede o escribe por coordenadas. **Nota:** Filas y columnas empiezan en 1.	Para posiciones calculadas.	**openpyxl** Devuelve Cell.
ws.append	`ws.append(['Ana',30])`	Añade una fila al final.	Para incorporar registros.	**openpyxl** Modifica el libro o la hoja.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
ws.iter_rows	`list(ws.iter_rows(min_row=2,values_only=True))`	Recorre filas como valores.	Para leer sin objetos Cell.	**openpyxl** Devuelve lista.
dataframe_to_rows	`list(dataframe_to_rows(df,index=False,header=True))`	Convierte pandas en filas de Excel.	Para rellenar una hoja desde pandas.	**openpyxl** Devuelve lista.
DataFrame desde hoja	`pd.DataFrame(ws.values)`	Convierte valores de una hoja en tabla. **Nota:** Separa la primera fila si contiene encabezados.	Para continuar con pandas.	**openpyxl** Devuelve DataFrame.
wb.save	`wb.save('informe.xlsx')`	Guarda el libro. **Nota:** Sobrescribe el archivo si existe.	Para entregar el Excel.	**openpyxl** Modifica el libro o la hoja.
wb.close	`wb.close()`	Cierra recursos del libro.	Para finalizar lecturas en modo read_only.	**openpyxl** Devuelve None.

openpyxl · Dar formato y preparar informes

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
Cell.number_format	`ws['C2'].number_format='#,##0.00'`	Asigna formato visual. **Nota:** No convierte texto en número.	Para mostrar decimales.	**openpyxl** Modifica el libro o la hoja.
Font	`ws['A1'].font=Font(bold=True,color='FFFFFF')`	Cambia la tipografía.	Para destacar encabezados.	**openpyxl** Modifica el libro o la hoja.
PatternFill	`ws['A1'].fill=PatternFill(fill_type='solid', fgColor='00838B')`	Colorea el fondo.	Para diferenciar títulos.	**openpyxl** Modifica el libro o la hoja.
Alignment	`ws['A1'].alignment=Alignment(horizontal= 'center')`	Alinea el contenido.	Para ordenar la presentación.	**openpyxl** Modifica el libro o la hoja.
ws.column_dimensions	`ws.column_dimensions['A'].width=22`	Ajusta ancho de columna.	Para evitar texto oculto.	**openpyxl** Modifica el libro o la hoja.
ws.freeze_panes	`ws.freeze_panes='A2'`	Fija filas o columnas al desplazarse.	Para mantener encabezados visibles.	**openpyxl** Modifica el libro o la hoja.
ws.auto_filter.ref	`ws.auto_filter.ref=ws.dimensions`	Activa un rango de autofiltro. **Nota:** No ejecuta el filtro en Python.	Para filtrar al abrir Excel.	**openpyxl** Modifica el libro o la hoja.
ws.merge_cells	`ws.merge_cells('H1:J1')`	Combina celdas. **Nota:** Solo la celda superior izquierda conserva un valor.	Para un título superior.	**openpyxl** Modifica el libro o la hoja.
Fórmula Excel	`ws['H2']='=SUM(F2:F13)'`	Escribe una fórmula. **Nota:** openpyxl no evalúa fórmulas.	Para recalcular al abrir en Excel.	**openpyxl** Modifica el libro o la hoja.
ColorScaleRule	`ws.conditional_formatting.add('F2:F13', ColorScaleRule(start_type='min', start_color='FFFFFF',end_type='max', end_color='00838B'))`	Añade escala de color condicional.	Para destacar importes altos.	**openpyxl** Modifica el libro o la hoja.

Función o atributo	Cómo se usa	Qué hace	Cuándo usarlo	Librería / resultado
Table Excel	`ws.add_table(ExcelTable(displayName='VentasTabla',ref='A1:F13'))`	Crea una tabla estructurada. **Nota:** Encabezados de texto y nombre único.	Para filtros y referencias de tabla.	**openpyxl** Modifica el libro o la hoja.
DataValidation	`v=DataValidation(type='list',formula1='"Sí,No"'); ws.add_data_validation(v); v.add('H2:H13')`	Añade validación de entrada.	Para limitar valores en Excel.	**openpyxl** Modifica el libro o la hoja.
BarChart	`graf=BarChart(); graf.add_data(Reference(ws, min_col=6,min_row=1,max_row=13), titles_from_data=True); ws.add_chart(graf,'H5')`	Inserta barras en la hoja.	Para un informe Excel con gráfico.	**openpyxl** Modifica el libro o la hoja.

Índice alfabético

Cada nombre lleva a su fila. Las variantes que comparten ficha remiten a la misma página.

Nombre · librería	Pág.	Nombre · librería	Pág.
accuracy_score · scikit-learn	31	stats.bootstrap · SciPy	24
wb.active · openpyxl	47	ax.boxplot · Matplotlib	13
rel.aggregate · DuckDB	40	sns.boxplot · Seaborn	19
Alignment · openpyxl	49	pc.cast · PyArrow	35
ax.annotate · Matplotlib	15	sns.catplot · Seaborn	20
ws.append · openpyxl	47	stats.norm.cdf · SciPy	24
np.arange · NumPy	7	ws.cell · openpyxl	47
np.argsort · NumPy	10	stats.chi2_contingency · SciPy	23
np.array · NumPy	7	rng.choice · NumPy	11
pa.array · PyArrow	34	classification_report · scikit-learn	31
np.asarray · NumPy	7	np.clip · NumPy	9
Asignación de celda · openpyxl	47	plt.close · Matplotlib	16
a.astype · NumPy	7	con.close · DuckDB	40
fig.autofmt_xdate · Matplotlib	15	wb.close · openpyxl	48
np.average · NumPy	10	coef_ · scikit-learn	30
ax.axhline · Matplotlib	14	sns.color_palette · Seaborn	20
ax.axvline · Matplotlib	14	fig.colorbar · Matplotlib	16
ax.bar · Matplotlib	13	ColorScaleRule · openpyxl	49
BarChart · openpyxl	50	ws.column_dimensions · openpyxl	49
ax.barh · Matplotlib	13	np.column_stack · NumPy	8
sns.barplot · Seaborn	18	rel.columns · DuckDB	40
engine.begin · SQLAlchemy	44	ColumnTransformer · scikit-learn	27

Nombre · librería	Pág.	Nombre · librería	Pág.
pa.concat_tables · PyArrow	34	rel.df · DuckDB	39
np.concatenate · NumPy	8	np.diff · NumPy	10
confusion_matrix · scikit-learn	31	sns.displot · Seaborn	20
duckdb.connect · DuckDB	39	engine.dispose · SQLAlchemy	45
engine.connect · SQLAlchemy	44	a.dtype · NumPy	7
a.copy · NumPy	8	DummyClassifier · scikit-learn	29
np.corrcoef · NumPy	10	DummyRegressor · scikit-learn	29
sns.countplot · Seaborn	18	sns.ecdfplot · Seaborn	18
CREATE TABLE AS · DuckDB	41	ax.errorbar · Matplotlib	13
create_engine · SQLAlchemy	44	con.execute · DuckDB	39
wb.create_sheet · openpyxl	47	Connection.execute · SQLAlchemy	45
cross_val_score · scikit-learn	32	EXPLAIN · DuckDB	42
sp.csr_array · SciPy	24	f1_score · scikit-learn	31
np.cumsum · NumPy	10	stats.f_oneway · SciPy	23
optimize.curve_fit · SciPy	24	feature_importances_ · scikit-learn	30
DataFrame desde hoja · openpyxl	48	ds.field · PyArrow	36
dataframe_to_rows · openpyxl	48	ax.fill_between · Matplotlib	14
ds.dataset · PyArrow	36	pc.fill_null · PyArrow	35
DataValidation · openpyxl	50	tabla.filter · PyArrow	34
np.random.default_rng · NumPy	10	rel.filter · DuckDB	39
stats.describe · SciPy	22	signal.find_peaks · SciPy	24
sns.despine · Seaborn	20	fit · scikit-learn	27

Nombre · librería	Pág.	Nombre · librería	Pág.
fit · scikit-learn	29	pf.iter_batches · PyArrow	36
Font · openpyxl	49	ws.iter_rows · openpyxl	48
ws.freeze_panes · openpyxl	49	JOIN · DuckDB	41
con.from_df · DuckDB	39	sns.jointplot · Seaborn	20
pa.Table.from_pandas · PyArrow	34	sns.kdeplot · Seaborn	18
np.full · NumPy	7	KMeans · scikit-learn	29
Fórmula Excel · openpyxl	49	stats.kruskal · SciPy	23
Inspector.get_columns · SQLAlchemy	45	Lasso · scikit-learn	28
get_feature_names_out · scikit-learn	27	ax.legend · Matplotlib	15
ax.grid · Matplotlib	15	stats.levene · SciPy	23
GridSearchCV · scikit-learn	32	rel.limit · DuckDB	40
sns.heatmap · Seaborn	19	LinearRegression · scikit-learn	28
ax.hist · Matplotlib	13	sns.lineplot · Seaborn	18
HistGradientBoostingRegressor · scikit-learn	28	stats.linregress · SciPy	23
sns.histplot · Seaborn	18	np.linspace · NumPy	7
ax.imshow · Matplotlib	13	load_workbook · openpyxl	47
inspect · SQLAlchemy	44	np.log1p · NumPy	10
stats.iqr · SciPy	22	LogisticRegression · scikit-learn	28
np.isclose · NumPy	10	np.linalg.lstsq · NumPy	11
np.isfinite · NumPy	9	make_column_selector · scikit-learn	27
np.isin · NumPy	10	make_pipeline · scikit-learn	27
np.isnan · NumPy	9	stats.mannwhitneyu · SciPy	23

Nombre · librería	Pág.	Nombre · librería	Pág.
np.mean · NumPy	9	stats.pearsonr · SciPy	22
mean_absolute_error · scikit-learn	31	permutation_importance · scikit-learn	32
np.median · NumPy	9	Pipeline · scikit-learn	27
ws.merge_cells · openpyxl	49	ax.plot · Matplotlib	13
pf.metadata · PyArrow	36	PolynomialFeatures · scikit-learn	27
MetaData · SQLAlchemy	45	stats.norm.ppf · SciPy	24
optimize.minimize · SciPy	24	precision_score · scikit-learn	31
MinMaxScaler · scikit-learn	26	predict · scikit-learn	29
np.nan_to_num · NumPy	9	predict_proba · scikit-learn	29
np.nanmean · NumPy	9	rel.project · DuckDB	39
rng.normal · NumPy	11	np.quantile · NumPy	10
tabla.num_rows · PyArrow	34	r2_score · scikit-learn	31
Cell.number_format · openpyxl	49	RandomForestClassifier · scikit-learn	29
OneHotEncoder · scikit-learn	26	RandomForestRegressor · scikit-learn	28
rel.order · DuckDB	40	a.ravel · NumPy	8
OrdinalEncoder · scikit-learn	26	con.read_csv · DuckDB	41
OVER · DuckDB	41	con.read_parquet · DuckDB	41
sns.pairplot · Seaborn	20	pq.read_schema · PyArrow	36
pq.ParquetFile · PyArrow	36	pd.read_sql_query · SQLAlchemy	44
PatternFill · openpyxl	49	pd.read_sql_table · SQLAlchemy	44
PCA · scikit-learn	29	pq.read_table · PyArrow	36
interpolate.PchipInterpolator · SciPy	24	recall_score · scikit-learn	31

Nombre · librería	Pág.	Nombre · librería	Pág.
ws.auto_filter.ref · openpyxl	49	ax.set · Matplotlib	15
con.register · DuckDB	39	set_output · scikit-learn	27
sns.regplot · Seaborn	19	sns.set_theme · Seaborn	18
sns.relplot · Seaborn	20	ax.set_xlim · Matplotlib	15
a.reshape · NumPy	7	ax.set_ylim · Matplotlib	15
sns.residplot · Seaborn	19	ax.set_yscale · Matplotlib	15
Ridge · scikit-learn	28	a.shape · NumPy	7
RobustScaler · scikit-learn	26	stats.shapiro · SciPy	23
roc_auc_score · scikit-learn	31	wb.sheetnames · openpyxl	47
root_mean_squared_error · scikit-learn	31	plt.show · Matplotlib	16
np.round · NumPy	10	SimpleImputer · scikit-learn	26
stats.norm.rvs · SciPy	24	tabla.slice · PyArrow	34
wb.save · openpyxl	48	tabla.sort_by · PyArrow	34
fig.savefig · Matplotlib	16	stats.spearmanr · SciPy	22
ax.scatter · Matplotlib	13	con.sql · DuckDB	39
sns.scatterplot · Seaborn	18	StandardScaler · scikit-learn	26
pa.schema · PyArrow	34	np.std · NumPy	9
tabla.schema · PyArrow	34	sns.stripplot · Seaborn	19
np.select · NumPy	9	plt.subplots · Matplotlib	13
tabla.select · PyArrow	34	np.sum · NumPy	10
select · SQLAlchemy	45	pc.sum · PyArrow	35
stats.sem · SciPy	22	func.sum · SQLAlchemy	45

Nombre · librería	Pág.	Nombre · librería	Pág.
Table · SQLAlchemy	45	stats.wilcoxon · SciPy	23
Table Excel · openpyxl	50	Workbook · openpyxl	47
text · SQLAlchemy	44	rel.write_csv · DuckDB	41
ax.tick_params · Matplotlib	15	ds.write_dataset · PyArrow	37
TimeSeriesSplit · scikit-learn	32	rel.write_parquet · DuckDB	41
tabla.to_pandas · PyArrow	34	pq.write_table · PyArrow	36
df.to_sql · SQLAlchemy	44	np.zeros · NumPy	7
dataset.to_table · PyArrow	36	stats.zscore · SciPy	22
train_test_split · scikit-learn	26		
transform · scikit-learn	27		
integrate.trapezoid · SciPy	24		
stats.trim_mean · SciPy	22		
stats.ttest_ind · SciPy	22		
stats.ttest_rel · SciPy	23		
rel.types · DuckDB	40		
np.unique · NumPy	10		
con.unregister · DuckDB	40		
ws[cell].value · openpyxl	47		
np.var · NumPy	9		
sns.violinplot · Seaborn	19		
wb[hoja] · openpyxl	47		
np.where · NumPy	9		